



CS2023 - Programación III
Ejercicio 2: patrones de diseño en C++20
Rubén Rivas Medina
Junio 2026

Ejercicio 2: Gastos personales

Se desea construir una herramienta de consola para revisar gastos personales del mes. Cada gasto tiene fecha, categoría, descripción y monto. El usuario debe poder ordenar la lista con distintos criterios y exportarla en el formato que necesite.

El diseño debe permitir agregar nuevos formatos de exportación, nuevos criterios de ordenamiento y pasos adicionales sobre la salida, como auditoría o resumen, sin modificar el flujo principal del programa.

La entrega se realizará en GitHub Classroom. El repositorio debe compilar y ejecutar sus pruebas automáticamente con GitHub Actions cada vez que el alumno haga push o abra un pull_request.

Patrones requeridos

Factory Method	Crea exportadores csv, json o text.
Decorator	Envuelve un exportador con AuditedExporter<T> o SummaryExporter<T>.
Strategy	Ordena gastos por fecha, categoría o monto.

Esqueleto C++20

```
struct Expense {
    std::string date;      // "2026-06-04"
    std::string category;  // "food", "transport", "books"
    std::string detail;
    double amount{};
};

using ExpenseList = std::vector<Expense>;

template<class T>
concept ExpenseExporter =
    requires(T exporter, const ExpenseList& expenses) {
        { exporter.export_expenses(expenses) } -> std::same_as<std::string>;
    };

using ExportFunction = std::function<std::string(const ExpenseList&)>;

template<class R, class Cmp>
concept SortStrategy =
    std::ranges::random_access_range<R> &&
    std::sortable<std::ranges::iterator_t<R>, Cmp>;

void sort_with(auto& values, auto cmp)
    requires SortStrategy<decltype(values), decltype(cmp)>;
```

```

template<ExpenseExporter E, class... Args>
ExportFunction make_exporter(Args&&... args);

struct CsvExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

struct JsonExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

struct TextExporter {
    std::string export_expenses(const ExpenseList& expenses) const;
};

template<ExpenseExporter Inner>
struct AuditedExporter {
    Inner inner;
    std::string export_expenses(const ExpenseList& expenses) const;
};

template<ExpenseExporter Inner>
struct SummaryExporter {
    Inner inner;
    std::string export_expenses(const ExpenseList& expenses) const;
};

```

Reglas mínimas

- El cliente no debe tener if/else repetidos para crear exportadores.
- La fábrica debe seguir el estilo de la diapositiva: `template<ExpenseExporter E, class... Args>`.
- `make_exporter<E>()` debe retornar un `ExportFunction` que llame a `E::export_expenses`.
- Los decoradores deben aceptar cualquier tipo que cumpla `ExpenseExporter`.
- Los decoradores deben poder encadenarse sin cambiar el código cliente.
- Las estrategias deben ser callable compatibles con `sort_with`.
- `sort_with` debe usar `std::ranges::sort`.
- No usar `new/delete` directos; usar valores, RAII y objetos estándar.

Ejemplo de composición:

```

auto exporter =
    AuditedExporter{
        SummaryExporter{
            CsvExporter{}
        }
    };

auto output = exporter.export_expenses(expenses);

```

Estructura esperada del repositorio

El repositorio de GitHub Classroom debe incluir como mínimo:

```

.
├── CMakeLists.txt
├── include/
│   └── expenses.h
├── src/
│   └── main.cpp
├── tests/
│   └── test_expenses.cpp
└── .github/
    └── workflows/
        └── tests.yml

```

El archivo `CMakeLists.txt` debe construir un ejecutable demostrativo y un ejecutable de pruebas. Un ejemplo mínimo es:

```

cmake_minimum_required(VERSION 3.22)
project(expenses_patterns LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(expenses_app
    src/main.cpp
)
target_include_directories(expenses_app PRIVATE include)

add_executable(expenses_tests
    tests/test_expenses.cpp
)
target_include_directories(expenses_tests PRIVATE include)

enable_testing()
add_test(NAME expenses_tests COMMAND expenses_tests)

```

Pipeline de GitHub Actions

Crear el archivo `.github/workflows/tests.yml` con un pipeline equivalente a:

```

name: C++20 tests

on:
  push:
    branches: [main, master]
  pull_request:
  workflow_dispatch:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

```

```

- name: Configure
  run: cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug

- name: Build
  run: cmake --build build --config Debug

- name: Run tests
  run: ctest --test-dir build --output-on-failure

```

El pipeline debe quedar en verde para considerar la entrega como compilable. Las pruebas pueden usar assert, pero deben ejecutarse desde ctest.

Pruebas sugeridas

Las siguientes pruebas son una guía. Pueden adaptarse siempre que evalúen los mismos contratos de diseño.

Strategy: orden por monto

```

void test_strategy_sort_by_amount_descending() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20},
        {"2026-06-03", "books", "cpp", 45.00}
    };

    auto by_amount = [](const Expense& a, const Expense& b) {
        return a.amount > b.amount;
    };

    sort_with(expenses, by_amount);

    assert(expenses.front().amount == 45.00);
    assert(expenses.back().amount == 3.20);
}

```

Strategy: orden por categoría y fecha

```

void test_strategy_sort_by_category_then_date() {
    ExpenseList expenses{
        {"2026-06-03", "transport", "bus", 3.20},
        {"2026-06-01", "books", "cpp", 45.00},
        {"2026-06-02", "books", "algorithms", 38.00}
    };

    auto by_category_then_date = [](const Expense& a, const Expense& b) {
        return std::tie(a.category, a.date) < std::tie(b.category, b.date);
    };

    sort_with(expenses, by_category_then_date);

    assert(expenses[0].category == "books");
    assert(expenses[0].date == "2026-06-01");
    assert(expenses[2].category == "transport");
}

```

Factory Method: creación de exportador CSV

```
void test_factory_create_csv_exporter() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    auto exporter = make_exporter<CsvExporter>();
    auto output = exporter(expenses);

    assert(output.contains("date,category,detail,amount"));
    assert(output.contains("food"));
    assert(output.contains("transport"));
}
```

Factory Method: creación de exportador JSON

```
void test_factory_create_json_exporter() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50}
    };

    auto exporter = make_exporter<JsonExporter>();
    auto output = exporter(expenses);

    assert(output.contains("\"category\""));
    assert(output.contains("\"food\""));
    assert(output.contains("\"amount\""));
}
```

Decorator: auditoría preserva la exportación interna

```
void test_decorator_audit_preserves_inner_export() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    AuditedExporter exporter{CsvExporter{}};
    auto output = exporter.export_expenses(expenses);

    assert(output.contains("date,category,detail,amount"));
    assert(output.contains("transport"));
    assert(output.contains("audit"));
}
```

Decorator: resumen agrega total

```
void test_decorator_summary_adds_total() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20}
    };

    SummaryExporter exporter{TextExporter{}};
```

```

    auto output = exporter.export_expenses(expenses);

    assert(output.contains("food"));
    assert(output.contains("total"));
    assert(output.contains("21.7") || output.contains("21.70"));
}

```

Integración: exportación completa de gastos

```

void test_integration_export_sorted_expenses() {
    ExpenseList expenses{
        {"2026-06-01", "food", "lunch", 18.50},
        {"2026-06-02", "transport", "bus", 3.20},
        {"2026-06-03", "books", "cpp", 45.00}
    };

    auto by_amount = [](const Expense& a, const Expense& b) {
        return a.amount > b.amount;
    };

    sort_with(expenses, by_amount);

    auto exporter =
        AuditedExporter{
            SummaryExporter{
                CsvExporter{}
            }
        };

    auto output = exporter.export_expenses(expenses);

    assert(expenses.front().amount == 45.00);
    assert(output.contains("date,category,detail,amount"));
    assert(output.contains("total"));
    assert(output.contains("audit"));
}

```

El archivo tests/test_expenses.cpp debe llamar todas las pruebas desde main:

```

int main() {
    test_strategy_sort_by_amount_descending();
    test_strategy_sort_by_category_then_date();
    test_factory_create_csv_exporter();
    test_factory_create_json_exporter();
    test_decorator_audit_preserves_inner_export();
    test_decorator_summary_adds_total();
    test_integration_export_sorted_expenses();
}

```

Entregables

- Código C++20 separado en headers claros.
- main.cpp con un flujo demostrativo de carga, ordenamiento y exportación.
- tests/test_expenses.cpp con al menos 5 pruebas ejecutadas por ctest.
- .github/workflows/tests.yml con el pipeline de GitHub Actions.

- Captura o enlace al resultado exitoso del workflow en GitHub Classroom.
- Breve respuesta en README.md: ¿qué patrón desacopla qué parte del diseño?

Evaluación

Criterio	Puntaje
Patrones pedidos aplicados correctamente	6 pts
Uso idiomático de C++20: concepts, templates, ranges, RAII	4 pts
Diseño desacoplado y componentes pequeños	4 pts
Pruebas automatizadas ejecutadas por GitHub Actions	4 pts
Explicación breve de decisiones de diseño	2 pts